

```
// ===== TASK 1: MIN HEAP =====
```

```
public class MinHeap {

    private Integer[] heap;    // Heap array
    private int size;         // Current elements
    private int capacity;     // Maximum elements

    public MinHeap(int capacity) {
        this.capacity = capacity; // Set capacity
        heap = new Integer[capacity + 1]; // 1-based array
        size = 0;                // Initially empty
    }

    // INSERT
    public void insert(int value) {

        if (size >= capacity) { // Check full
            System.out.println("Heap is full");
            return;
        }

        size++;                // Increase size
        heap[size] = value;    // Add at end
        heapifyUp(size);       // Fix upward
    }

    // HEAPIFY UP
    private void heapifyUp(int index) {

        while (index > 1) {    // Until root

            int parent = index / 2; // Find parent

            if (heap[parent] > heap[index]) { // Parent bigger → swap

                int temp = heap[parent]; // Save parent
                heap[parent] = heap[index]; // Child → parent
                heap[index] = temp;       // Parent → child

                index = parent;           // Move upward
            }
            else {
                break;                    // Heap is correct
            }
        }
    }
}
```

```

    }
}
}

```

// EXTRACT MIN

```

public Integer extractMin() {

    if (size == 0) {          // Empty?
        return null;         // Nothing to remove
    }

    int min = heap[1];        // Root = minimum

    heap[1] = heap[size];    // Last → root
    heap[size] = null;       // Remove last
    size--;                  // Decrease size

    heapifyDown(1);          // Fix downward

    return min;              // Return minimum
}

```

// HEAPIFY DOWN

```

private void heapifyDown(int index) {

    while (index * 2 <= size) { // Has child?

        int left = index * 2;   // Left child
        int right = index * 2 + 1; // Right child

        int smaller = left;     // Assume left smaller

        if (right <= size && heap[right] < heap[left]) {
            smaller = right;    // Right is smaller
        }

        if (heap[index] > heap[smaller]) { // Current bigger → swap

            int temp = heap[index];    // Save current
            heap[index] = heap[smaller]; // Child → current
            heap[smaller] = temp;       // Current → child

            index = smaller;            // Move downward
        }
    }
}

```

```

        else {
            break;           // Heap is correct
        }
    }
}

// PEEK
public Integer peek() {

    if (size == 0) {        // Empty?
        return null;
    }

    return heap[1];         // Return minimum
}

// ISEMPY
public boolean isEmpty() {

    return size == 0;       // True if empty
}
}

// ===== TASK 2: MAX HEAP =====

public class MaxHeap {

    private Integer[] heap;    // Heap array
    private int size;         // Current elements
    private int capacity;     // Maximum elements

    public MaxHeap(int capacity) {
        this.capacity = capacity; // Set capacity
        heap = new Integer[capacity + 1]; // 1-based
        size = 0;             // Initially empty
    }

    // INSERT
    public void insert(int value) {

        if (size >= capacity) { // Check full
            System.out.println("Heap is full");
            return;
        }
    }
}

```

```

size++;           // Increase size
heap[size] = value; // Add at end
heapifyUp(size);   // Fix upward
}

```

// HEAPIFY UP

```

private void heapifyUp(int index) {

    while (index > 1) {           // Until root

        int parent = index / 2; // Find parent

        if (heap[parent] < heap[index]) { // Parent smaller → swap

            int temp = heap[parent]; // Save parent
            heap[parent] = heap[index]; // Child → parent
            heap[index] = temp;        // Parent → child

            index = parent;           // Move upward
        }
        else {
            break;                   // Heap correct
        }
    }
}

```

// EXTRACT MAX

```

public Integer extractMax() {

    if (size == 0) {           // Empty?
        return null;
    }

    int max = heap[1];         // Root = maximum

    heap[1] = heap[size];      // Last → root
    heap[size] = null;         // Remove last
    size--;                    // Decrease size

    heapifyDown(1);           // Fix downward

    return max;                // Return maximum
}

```

// HEAPIFY DOWN

```
private void heapifyDown(int index) {  
  
    while (index * 2 <= size) { // Has a child?  
  
        int left = index * 2; // Left child  
        int right = index * 2 + 1; // Right child  
  
        int larger = left; // Assume left larger  
  
        if (right <= size && heap[right] > heap[left]) {  
            larger = right; // Right is larger  
        }  
  
        if (heap[index] < heap[larger]) { // Current smaller → swap  
  
            int temp = heap[index]; // Save current  
            heap[index] = heap[larger]; // Child → current  
            heap[larger] = temp; // Current → child  
  
            index = larger; // Move downward  
        }  
        else {  
            break; // Heap correct  
        }  
    }  
}
```

// PEEK

```
public Integer peek() {  
  
    if (size == 0) { // Empty?  
        return null;  
    }  
  
    return heap[1]; // Return maximum  
}
```

// ISEMPY

```
public boolean isEmpty() {  
  
    return size == 0; // True if empty  
}
```

```
}
```

```
// ===== TASK 3: HEAP SORT =====
```

```
// Ascending order → MinHeap
```

```
public static void outPlaceHeapsort(int[] arr) {
```

```
    MinHeap heap = new MinHeap(arr.length); // Create heap
```

```
    for (int i = 0; i < arr.length; i++) {  
        heap.insert(arr[i]);          // Insert all  
    }
```

```
    for (int i = 0; i < arr.length; i++) {  
        arr[i] = heap.extractMin();    // Smallest → array  
    }  
}
```

```
// ===== TASK 4: MACHINE TASK =====
```

```
public static int[] taskScheduler(int[] tasks, int m) {
```

```
    MinHeap heap = new MinHeap(m);    // Create MinHeap
```

```
    for (int i = 0; i < m; i++) {  
        heap.insert(0);                // Every machine load = 0  
    }
```

```
    for (int i = 0; i < tasks.length; i++) {  
        int load = heap.extractMin();  // Get smallest load
```

```
        load = load + tasks[i];        // Add task time
```

```
        heap.insert(load);             // Put updated load back  
    }
```

```
    int[] result = new int[m];         // Answer array
```

```
    for (int i = 0; i < m; i++) {  
        result[i] = heap.extractMin(); // Get final loads  
    }
```

```

    return result;          // Return answer
}

```

// ===== TASK 5: TOP K LARGEST =====

```

public static int[] kth(int[] nums, int k) {

    MaxHeap heap = new MaxHeap(nums.length); // Create MaxHeap

    for (int i = 0; i < nums.length; i++) {
        heap.insert(nums[i]);          // Insert all numbers
    }

    int[] result = new int[k];          // Answer array

    for (int i = 0; i < k; i++) {
        result[i] = heap.extractMax();    // Get largest
    }

    return result;          // Return top k
}

```

// ===== TASK 6: PRIORITY QUEUE =====

```

// Stores task name + priority
class Task {

    String name;          // Task name
    int priority;          // Task priority

    Task(String name, int priority) {
        this.name = name;    // Set name
        this.priority = priority; // Set priority
    }
}

```

```

// CUSTOM MAX HEAP
class PriorityMaxHeap {

```

```

    private Task[] heap;    // Task array

```



```
}  
}
```

```
// EXTRACT MAX
```

```
public Task extractMax() {
```

```
    if (size == 0) {          // Empty?  
        return null;  
    }
```

```
    Task max = heap[1];      // Highest priority
```

```
    heap[1] = heap[size];    // Last → root  
    heap[size] = null;       // Remove last  
    size--;                  // Decrease size
```

```
    heapifyDown(1);          // Fix downward
```

```
    return max;              // Return task  
}
```

```
// HEAPIFY DOWN
```

```
private void heapifyDown(int index) {
```

```
    while (index * 2 <= size) {    // Has child?
```

```
        int left = index * 2;      // Left child  
        int right = index * 2 + 1; // Right child
```

```
        int larger = left;          // Assume left larger
```

```
        if (right <= size &&  
            heap[right].priority > heap[left].priority) {
```

```
            larger = right;          // Right has higher priority  
        }
```

```
        if (heap[index].priority < heap[larger].priority) {  
            // Current priority smaller → swap
```

```
            Task temp = heap[index]; // Save current  
            heap[index] = heap[larger]; // Child → current  
            heap[larger] = temp;        // Current → child
```

```

        index = larger;          // Move downward
    }
    else {
        break;                  // Heap correct
    }
}
}

public boolean isEmpty() {

    return size == 0;          // Check empty
}
}

// TASK SCHEDULER
public static String[] scheduleTasks(
    String[] task_names,
    int[] priorities) {

    PriorityMaxHeap heap =
        new PriorityMaxHeap(task_names.length);
    // Create priority heap

    for (int i = 0; i < task_names.length; i++) {

        heap.insert(
            task_names[i],
            priorities[i]
        );          // Insert task + priority
    }

    String[] result =
        new String[task_names.length];  // Answer array

    for (int i = 0; i < result.length; i++) {

        Task task = heap.extractMax();  // Highest priority

        result[i] = task.name;          // Store task name
    }

    return result;                    // Return ordered tasks
}

```

